ColumnTransformer

Follow *Introduction to Machine Learning* Chapter 4

- Section 4.3 Convenient ColumnTransformer (p.224)

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

In [ ]:
```
import mglearn
```

In [ ]:
```
import os
# The file has no headers naming the columns, so we pass header=None
# and provide the column names explicitly in "names"
adult_path = os.path.join(mglearn.datasets.DATA_PATH, "adult.data")
data = pd.read_csv(    adult_path,
    header=None,
    index_col=False,
    skipinitialspace=True, #remove space after comma
    names=['age', 'workclass', 'fnlwgt', 'education',  'education-num',
           'marital-status', 'occupation', 'relationship', 'race', 'gender',
           'capital-gain', 'capital-loss', 'hours-per-week', 'native-country',
           'income'])
# For illustration purposes, we only select some of the columns
data = data[['age', 'workclass', 'education', 'gender', 'hours-per-week',
             'occupation', 'income']]
# IPython.display allows nice output formatting within the Jupyter notebook
display(data.head())
```

Build the ColumnTransformer

In [ ]:
```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

ct = ColumnTransformer(
    [("scaling", StandardScaler(), ['age', 'hours-per-week']),
     ("onehot", OneHotEncoder(sparse_output=False), ['workclass', 'education', 'gender
```

In [ ]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
# get all columns apart from income for the features
data_features = data.drop("income", axis=1)
# split dataframe and income
X_train, X_val, y_train, y_val = train_test_split(
    data_features, data.income, random_state=0)

ct.fit(X_train)
X_train_trans = ct.transform(X_train)
print(X_train_trans.shape)
```

Train the model using transformed data

Note that validation data `X_val` needs to be transformed with the learned transformer too.

In [ ]:
```
logreg = LogisticRegression(max_iter=1000)
logreg.fit(X_train_trans, y_train)

X_val_trans = ct.transform(X_val)
print("Validation score: {:.2f}".format(logreg.score(X_val_trans, y_val)))
```

Access ColumnTransformer components

In [ ]:
```
ct.named_transformers_.onehot
```

In [ ]:
```
ct.named_transformers_.onehot.get_feature_names_out()
```

Convenience function: `make_column_transformer()`

In [ ]:
```
from sklearn.compose import make_column_transformer
ct = make_column_transformer(
    (StandardScaler(), ['age', 'hours-per-week']),
    (OneHotEncoder(sparse=False), ['workclass', 'education', 'gender', 'occupation']))
```

In [ ]:
```
ct.fit(X_train)
```

In [ ]:
```
ct.named_transformers_
```

Excercise: Apply ColumnTransformer to heart disease data

In [ ]:
```
def load_heart_disease():
    '''Load and pre-process heart disease data

    if processed.hungarian.data file is not present.

    it will be downloaded from
    https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/processed.

    return: data(DataFrame)

    '''

    import os
    import requests


    file_url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/heart-diseas
    file_name = file_url.split('/')[-1]

    if not os.path.isfile(file_name):
        print('Downloading from {}'.format(file_url))
```

```
        r = requests.get(file_url)
        with open(file_name,'wb') as output_file:
            output_file.write(r.content)

    data = pd.read_csv(file_name,
                   na_values='?',
                   names=[ 'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs',
                           'restecg', 'thalach', 'exang', 'oldpeak', 'slope',
                           'ca', 'thal', 'num'])

    # drop columns with many missing data
    data = data.drop(columns=['slope', 'ca', 'thal'])

    # fill in remaining missing data with mean() per column
    data = data.fillna(data.mean())

    return data
```

In [ ]:
```
data = load_heart_disease()
```

In [ ]:
```
data.head()
```

In [ ]:
```
data.describe()
```

Which columns are numerical (quantitative), which are categorical (qualitative)?

Consult the data description, or use `value_counts()` to guess.

In [ ]:
```
data.cp.value_counts()
```

By using the mean to fill in NaN, we made a mistake for the `restecg` column:

In [ ]:
```
data.restecg.value_counts()
```

Let's fix this:

In [ ]:
```
# Pandas where function replaces every value that does not satisfy the condition with
data.restecg = data.restecg.where(data.restecg >= 1, 0)
```

In [ ]:
```
data.restecg.value_counts()
```

In [ ]:
```
# TODO: which columns to scale, onehot or do nothing?
ct = ColumnTransformer(
    [("scaling", StandardScaler(), ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']),
     ("onehot", OneHotEncoder(sparse_output=False), ['cp', 'restecg']),
    ("nothing", 'passthrough', ['sex', 'fbs', 'exang'])])
```

In [ ]:
```
# get all columns apart from num for the features
X = data.drop(columns='num')
y = data['num']
print(X.shape)
print(y.shape)
```

```
# split dataframe and income
X_train, X_val, y_train, y_val = train_test_split(X, y,
                                 test_size=0.2, stratify=y,random_state=31)

ct.fit(X_train)
X_train_trans = ct.transform(X_train)
print(X_train_trans.shape)
```

In [ ]:
```
logreg = LogisticRegression(max_iter=1000)
logreg.fit(X_train_trans, y_train)

X_val_trans = ct.transform(X_val)
print("Train score: {:.2f}".format(logreg.score(X_train_trans, y_train)))
print("Validation score: {:.2f}".format(logreg.score(X_val_trans, y_val)))
```

Are we overfitting? Let's try and reduce complexity by increasing regularization - reduce C:

In [ ]:
```
logreg = LogisticRegression(C=0.01,max_iter=1000)
logreg.fit(X_train_trans, y_train)

X_val_trans = ct.transform(X_val)
print("Train score: {:.2f}".format(logreg.score(X_train_trans, y_train)))
print("Validation score: {:.2f}".format(logreg.score(X_val_trans, y_val)))
```

Compare that to the unscaled dataset:

In [ ]:
```
logreg = LogisticRegression(max_iter=1000)
logreg.fit(X_train, y_train)

print("Train score: {:.2f}".format(logreg.score(X_train, y_train)))
print("Validation score: {:.2f}".format(logreg.score(X_val, y_val)))
```

The linear model is now much more flexible, it gained some non-linearity, similar to decision tree.

In [ ]:
```

```